

Week 11 Summary:

Inductive/Imperative Programming: smaller set of instructions , detailing

```
s = []
```

```
List = [1,2,3,4,5]
```

```
foreach i in List{
```

```
    if (i % 2 == 0){
```

```
        S = s ++ [i]
```

```
    }
```

```
}
```

Descriptive/Declarative Programming: focused on what to achieve , output oriented , No implementation details

```
S = [i for i in List if(i % 2 == 0)]
```

SQL – Select score from students where id < 100

Recursion:

Recursion is a process in which a procedure calls itself.

Only those problems which have a base condition can be solved using recursion.

Need of recursion: reduce length of code, easy to write and read

Memory management: Uses LIFO(Last in first out), like stack data structure, maintain activation record

Example: Countdown(5)

```
void Countdown(int n) {  
    for (i=n;i>0;i--)  
        cout<<i<<" ";  
    cout<<"\n";  
}
```

Without Recursion

```
void CountDown(int n) {
    cout<<n<<" ";
    CountDown(n-1);
}
```

Runs Infinitely

```
void CountDown(int n) {
    if(n>0) {
        cout<<n<<" ";
        CountDown(n-1);
    }
}
```

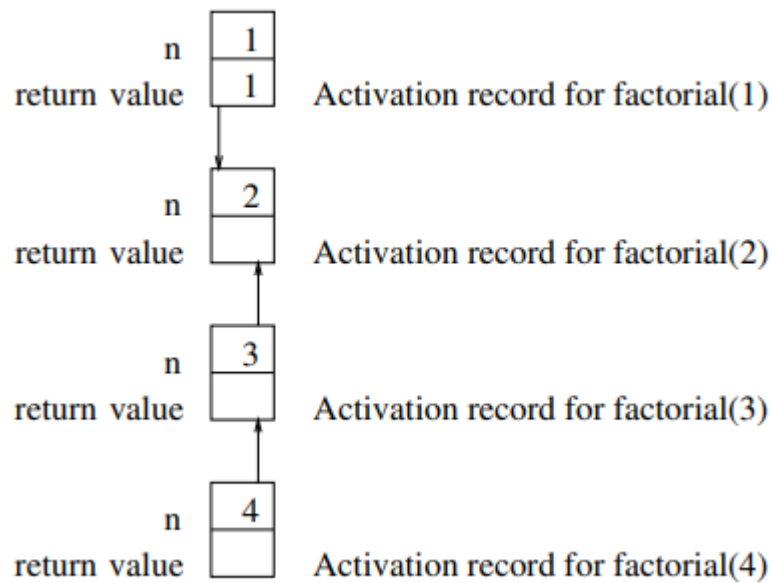
With recursion

Factorial of a number :

```
Procedure factorial (n)
    fact = 1
    while (n > 0) {
        fact = fact * n
        n = n - 1
    }
    return (fact)
End factorial
```

```
Procedure Factorial(n)
    if (n == 0) {
        return(1)
    }
    else {
        return(n * factorial(n-1))
    }
End Factorial
```

Base condition



Fibonacci Numbers:

$$\text{Fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{if } n > 1 \end{cases}$$

```
int Fib(int N) {
    if (N <= 1)
        return(N);
    else
        return(Fib(N-1) + Fib(N-2));
}
```

$\text{Fib}(0) = 0$
 $\text{Fib}(1) = 1$
 $\text{Fib}(2) = \text{Fib}(0) + \text{Fib}(1) = 0 + 1 = 1$
 $\text{Fib}(3) = \text{Fib}(1) + \text{Fib}(2) = 1 + 1 = 2$
 $\text{Fib}(4) = \text{Fib}(2) + \text{Fib}(3) = 1 + 2 = 3$
 $\text{Fib}(5) = \text{Fib}(3) + \text{Fib}(4) = 2 + 3 = 5$
 $\text{Fib}(6) = \text{Fib}(4) + \text{Fib}(5) = 3 + 5 = 8$
 $\text{Fib}(7) = \text{Fib}(5) + \text{Fib}(6) = 5 + 8 = 13$
 etc.

Recursive Programming on Lists:

```

Procedure Listsum(l)
  if (l == []) {
    return(0)
  }
  else {
    return(first(l) +
           Listsum(rest(l)))
  }
End Listsum
  
```

Insertion sort:

```

Procedure InsertionSort(l)
  if (length(l) <= 1) {
    return(l)
  }
  else {
    return(
      SortedListInsert(
        InsertionSort(rest(l)),
        first(l)
      )
    )
  }
End InsertionSort
  
```

```

Procedure SortedListInsert(l,x)
  newList = []
  inserted = False

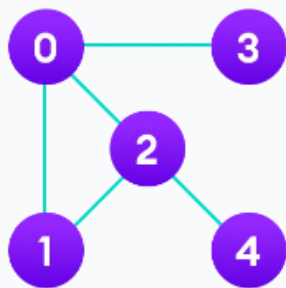
  foreach z in l {
    if (not(inserted)) {
      if (x < z) {
        newList = newList ++ [x]
        inserted = True
      }
    }
    newList = newList ++ [z]
  }

  if (not(inserted)) {
    newList = newList ++ [x]
  }

  return(newList)
End SortedListInsert

```

Depth first search:

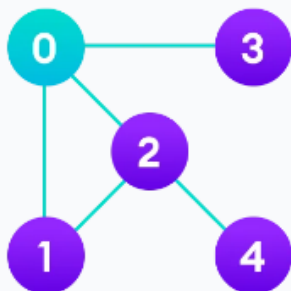


--	--	--	--	--

Visited

--	--	--	--	--

Stack

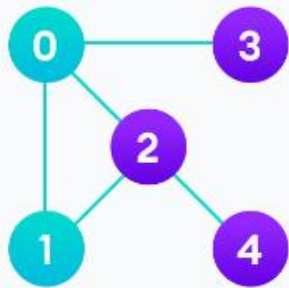


0				
---	--	--	--	--

Visited

1	2	3		
---	---	---	--	--

Stack

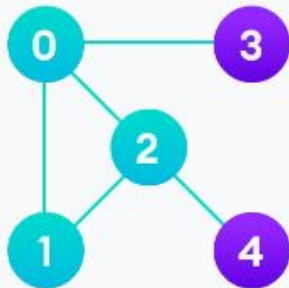


0	1			
---	---	--	--	--

Visited

2	3			
---	---	--	--	--

Stack

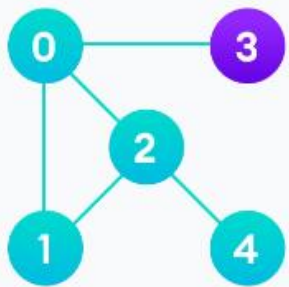


0	1	2		
---	---	---	--	--

Visited

4	3			
---	---	--	--	--

Stack

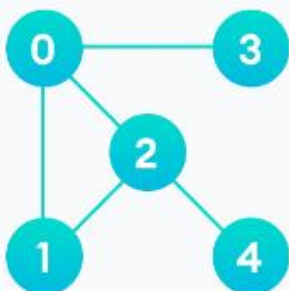


0	1	2	4	
---	---	---	---	--

Visited

3				
---	--	--	--	--

Stack



0	1	2	4	3
---	---	---	---	---

Visited

--	--	--	--	--

Stack

```

Procedure DFS(graph,visited,i)
  visited[i] = True
  foreach j in columns(graph){
    if (graph[i][j] == 1 and
        not(isKey(visited,j))) {
      visited =
        DFS(graph,visited,j)
    }
  }
  return(visited)
End DFS

```

